



جامعة عفت
EFFAT UNIVERSITY

CS 3081 - Artificial Intelligence

Assignment#2 - Fall 2024

Maram Alhusami

Instructor: **Akila Sarirete**

Date Last Edited: October 9, 2024

Contents

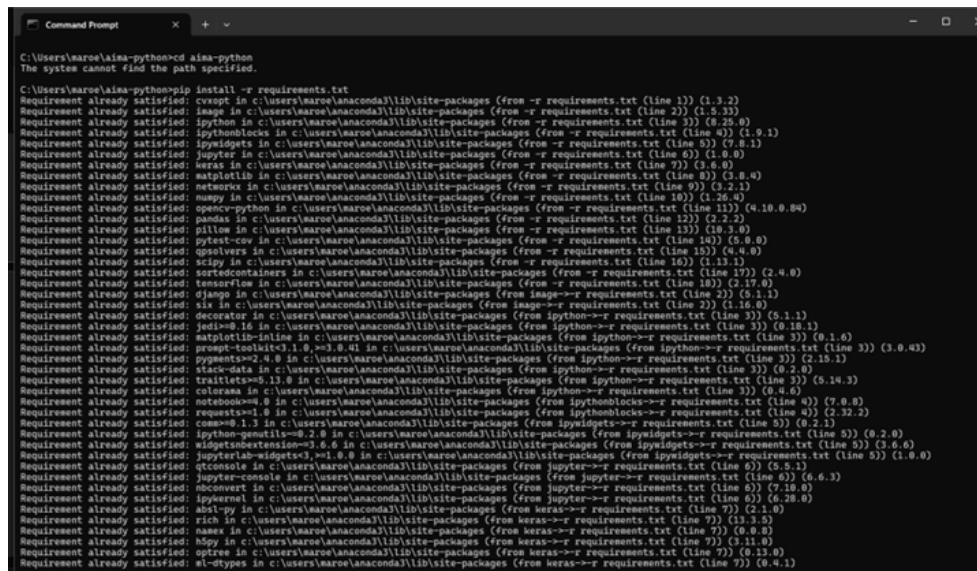
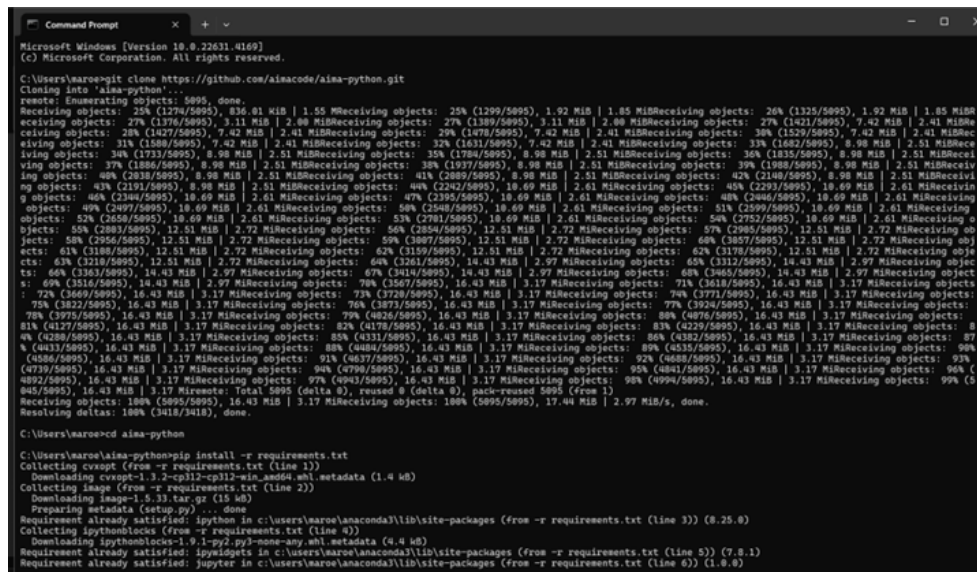
1	Introduction	2
2	Installation Steps	3
2.1	Installation of Python	3
2.2	Installation of Notebook	4
3	Breadth-First Search (BFS)	5
3.1	Visualization	5
3.2	Path Found:	5
3.3	The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:	5
3.4	The Output:	8
3.5	Output Clarification	8
3.6	Analysis	8
4	Depth-First Search (DFS)	10
4.1	Visualization	10
4.2	Path Found:	10
4.3	The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:	10
4.4	The Output:	12
4.5	Output Clarification	13
4.6	Analysis	13
5	Uniform Cost Search (UCS)	14
5.1	Visualization	14
5.2	Path Found:	14
5.3	The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:	14
5.4	The Output:	15
5.5	Output Clarification	15
5.6	Analysis	15
6	A* Search	16
6.1	Visualization	16
6.2	Path Found	16
6.3	The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:	16
6.4	The Output:	17
6.5	Output Clarification	17
6.6	Analysis	17
7	Comparison Table	18
8	Overall Comparison	18
9	Knowledge Acquired	18

1 Introduction

In the field of artificial intelligence, search algorithms play a crucial role in navigating complex problems and finding optimal solutions. This assignment focuses on the Romania Map Problem, where the goal is to identify the shortest path between two cities—such as Arad and Bucharest—using various search strategies.

This assignment will discuss the installation steps for the necessary software tools, provide an in-depth analysis of each search algorithm, and include a comparison table summarizing their performance based on empirical data. The sections that follow will provide a structured examination of these algorithms, highlighting their applicability to the Romania Map Problem and drawing conclusions about their effectiveness in various search scenarios.

2.1 Installation of Python



```

C:\Users\maroe\anaconda\python>git submodule init
Submodule 'aina-data' (https://github.com/aina-code/aina-data.git) registered for path 'aina-data'

C:\Users\maroe\anaconda\python>git submodule update
Cloning into 'c:/Users/maroe/anaconda/python/aina-data'...
Submodule path 'aina-data': checked out 'f6cbeaf1adb2c1cd07be826d17af5a8d3a7c2c86'

C:\Users\maroe\anaconda\python>pip install pytest
Requirement already satisfied: pytest in c:\users\maroe\anaconda\lib\site-packages (7.4.4)
Requirement already satisfied: iniconfig in c:\users\maroe\anaconda\lib\site-packages (from pytest) (1.1.1)
Requirement already satisfied: packaging in c:\users\maroe\anaconda\lib\site-packages (from pytest) (23.2)
Requirement already satisfied: pluggy<2.0,>=0.12 in c:\users\maroe\anaconda\lib\site-packages (from pytest) (1.0.0)
Requirement already satisfied: colorama in c:\users\maroe\anaconda\lib\site-packages (from pytest) (0.4.6)

C:\Users\maroe\anaconda\python>py.test
===== Test session starts =====
platform win32 -- Python 3.12.4, pytest-7.4.4, pluggy-1.8.0
rootdir: C:\Users\maroe\anaconda\python
configfile: pytest.ini
plugins: anyio=4.2.0, cov=5.0.0
collected 419 items

tests/test_agents.py ..... [ 3%]
tests/test_agents4e.py ..... [ 7%]
tests/test_csp.py ..... [14%]
tests/test_deep_learning4e.py ..... [18%]
tests/test_games4e.py ..... [19%]
tests/test_omni4e.py ..... [22%]
tests/test_learning.py ..... [24%]
tests/test_learning4e.py ..... [27%]
tests/test_logic.py ..... [34%]
tests/test_logic4e.py ..... [38%]
tests/test_md4e.py ..... [47%]
tests/test_md4e.py ..... [50%]
tests/test_md4e.py ..... [54%]
tests/test_md4e.py ..... [57%]
tests/test_perception4e.py ..... [60%]
tests/test_planning.py ..... [63%]
tests/test_probabilistic_learning.py ..... [67%]
tests/test_probability.py ..... [73%]
tests/test_probability4e.py ..... [79%]

```

Figure 3: Installing Project Dependencies

2.2 Installation of Notebook

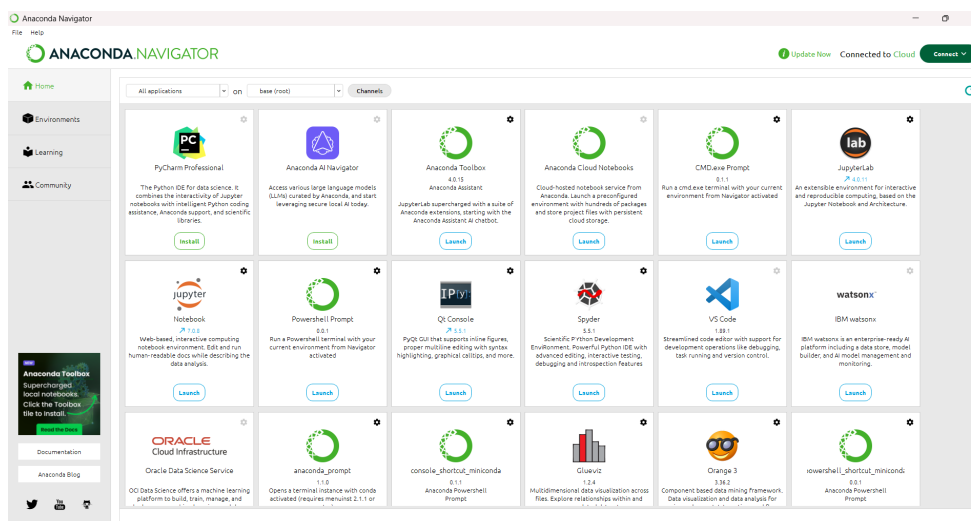
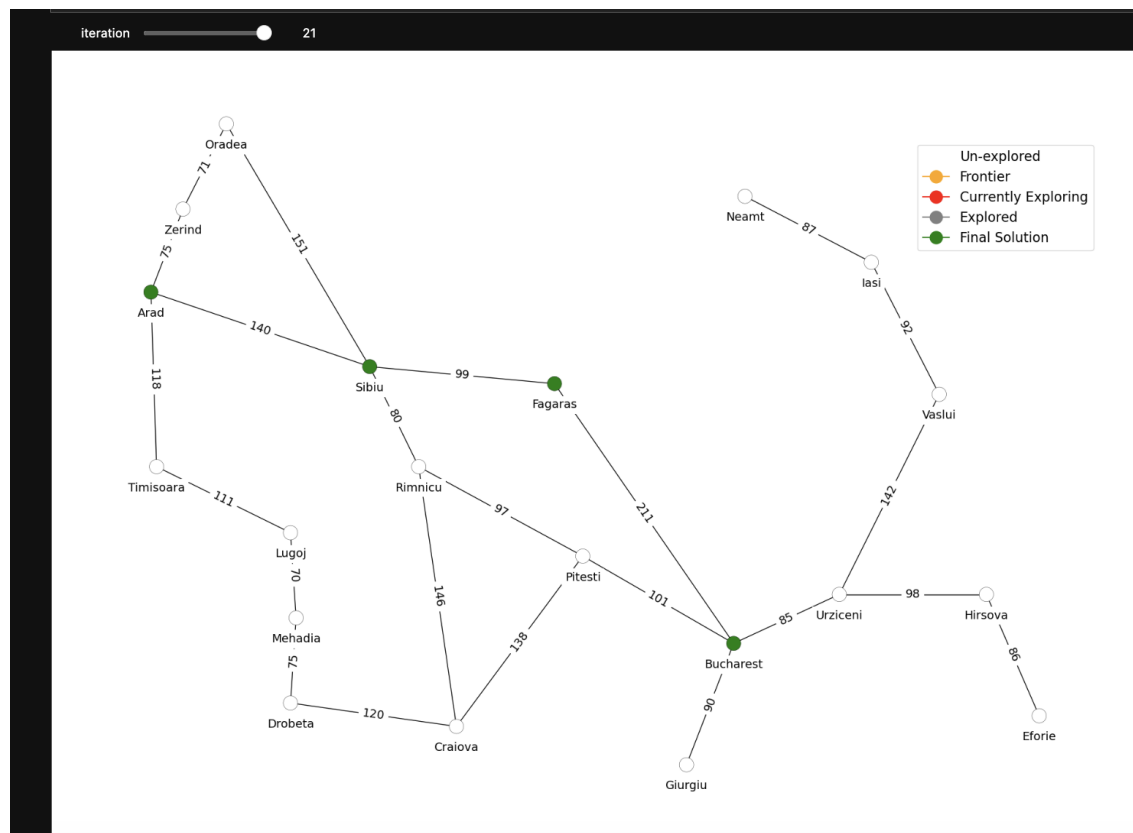


Figure 4: Launch Jupyter

3 Breadth-First Search (BFS)

3.1 Visualization



3.2 Path Found:

Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest

Explanation: BFS explores all nodes at the current depth level before moving to the next. It guarantees finding the shortest path in terms of the number of edges, but it doesn't consider travel costs. In this case, BFS finds a relatively straightforward route from Arad to Bucharest. However, while the number of edges is minimal, the total path cost may not be optimal due to varying road distances.

3.3 The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:

```

1 import time
2 from collections import deque
3
4 class Node:
5     def _init_(self, state, path_cost=0):
6         self.state = state
7         self.path_cost = path_cost # Initialize path cost
8
9     def expand(self, problem):
10         """Expand the node to generate children."""
11         children = []
12         for action, cost in problem.successors(self.state).items():
13             child_state = action

```

```

14         child_path_cost = self.path_cost + cost # Update the path cost for the child
15         children.append(Node(child_state, child_path_cost))
16     return children
17
18 class GraphProblem:
19     def _init_(self, initial, goal, graph):
20         self.initial = initial
21         self.goal = goal
22         self.graph = graph
23
24     def goal_test(self, state):
25         """Check if the current state is the goal state."""
26         return state == self.goal
27
28     def successors(self, state):
29         """Returns a dictionary of actions and costs to reach child states."""
30         return self.graph.get(state, {})
31
32 def breadth_first_search_graph(problem):
33     """Perform breadth-first search on the graph."""
34
35     # Metrics initialization
36     iterations = 0
37     all_node_colors = []
38     node_colors = {k: 'white' for k in problem.graph.keys()}
39
40     # Initialize the first node
41     node = Node(problem.initial)
42     node_colors[node.state] = "red"
43     iterations += 1
44     all_node_colors.append(dict(node_colors))
45
46     if problem.goal_test(node.state):
47         node_colors[node.state] = "green"
48         iterations += 1
49         all_node_colors.append(dict(node_colors))
50         return {
51             'nodes_expanded': 1,
52             'path_cost': node.path_cost,
53             'time_complexity': iterations,
54             'all_node_colors': all_node_colors,
55             'goal_node': node
56         }
57
58     frontier = deque([node])
59     node_colors[node.state] = "orange"
60     iterations += 1
61     all_node_colors.append(dict(node_colors))
62
63     explored = set()
64
65     # Start timing the search
66     start_time = time.time()
67
68     while frontier:
69         node = frontier.popleft()
70         node_colors[node.state] = "red"
71         iterations += 1
72         all_node_colors.append(dict(node_colors))
73
74         explored.add(node.state)
75
76         for child in node.expand(problem):
77             if child.state not in explored and child not in frontier:
78                 if problem.goal_test(child.state):
79                     node_colors[child.state] = "green"
80                     iterations += 1
81                     all_node_colors.append(dict(node_colors))

```

```

82         time_elapsed = time.time() - start_time # Calculate time taken
83         return {
84             'nodes_expanded': len(all_node_colors), # Count of nodes expanded
85             'path_cost': child.path_cost, # Path cost to the goal
86             'time_complexity': time_elapsed, # Time taken for the search
87             'all_node_colors': all_node_colors,
88             'goal_node': child
89         }
90     frontier.append(child)
91     node_colors[child.state] = "orange"
92     iterations += 1
93     all_node_colors.append(dict(node_colors))
94
95     node_colors[node.state] = "gray"
96     iterations += 1
97     all_node_colors.append(dict(node_colors))
98
99     return None
100
101 # Define the Romania problem
102 romania_map = {
103     'Arad': {'Sibiu': 140, 'Timisoara': 118},
104     'Sibiu': {'Arad': 140, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
105     'Timisoara': {'Arad': 118, 'Lugoj': 111},
106     'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
107     'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97},
108     'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101},
109     'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90},
110     'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
111     'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
112     'Drobeta': {'Mehadia': 75, 'Craiova': 120},
113     'Craiova': {'Drobeta': 120, 'Pitesti': 138},
114 }
115
116 # Instantiate the problem
117 romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)
118
119 # Run Breadth First Search and get metrics
120 bfs_metrics = breadth_first_search_graph(romania_problem)
121 if bfs_metrics:
122     print("Breadth First Search Metrics:")
123     print("Nodes Expanded:", bfs_metrics['nodes_expanded'])
124     print("Path Cost:", bfs_metrics['path_cost'])
125     print("Time Complexity:", bfs_metrics['time_complexity'])
126 else:
127     print("No solution found.")
128

```

Listing 1: Breadth First Search Algorithm

3.4 The Output:

```

frontier = deque([node])
node_colors[node.state] = "orange"
iterations += 1
all_node_colors.append(dict(node_colors))

explored = set()

# Start timing the search
start_time = time.time()

while frontier:
    node = frontier.popleft()
    node_colors[node.state] = "red"
    iterations += 1
    all_node_colors.append(dict(node_colors))

    explored.add(node.state)

    for child in node.expand(problem):
        if child.state not in explored and child not in frontier:
            if problem.goal_test(child.state):
                node_colors[child.state] = "green"
                iterations += 1
                all_node_colors.append(dict(node_colors))

                time_elapsed = time.time() - start_time # Calculate time taken
                return {
                    'nodes_expanded': len(all_node_colors), # Count of nodes expanded
                    'path_cost': child.path_cost, # Path cost to the goal
                    'time_complexity': time_elapsed, # Time taken for the search
                    'all_node_colors': all_node_colors,
                    'goal_node': child
                }
            frontier.append(child)
            node_colors[child.state] = "orange"
            iterations += 1
            all_node_colors.append(dict(node_colors))

    node_colors[node.state] = "gray"
    iterations += 1
    all_node_colors.append(dict(node_colors))

return None

# Define the Romania problem
romania_map = {
    'Arad': {'Sibiu': 140, 'Timisoara': 118},
    'Sibiu': {'Arad': 140, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Pitesti': 130},
}

# Instantiate the problem
romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)

# Run Breadth First Search and get metrics
bfs_metrics = breadth_first_search_graph(romania_problem)
if bfs_metrics:
    print("Breadth First Search Metrics:")
    print("Nodes Expanded:", bfs_metrics['nodes_expanded'])
    print("Path Cost:", bfs_metrics['path_cost'])
    print("Time Complexity:", bfs_metrics['time_complexity'])
else:
    print("No solution found.")

Breadth First Search Metrics:
Nodes Expanded: 15
Path Cost: 450
Time Complexity: 4.100799560546875e-05

```

3.5 Output Clarification

- Nodes Expanded: 15
- Total Cost: 450
- Time Complexity: 4.100799560546875e-05

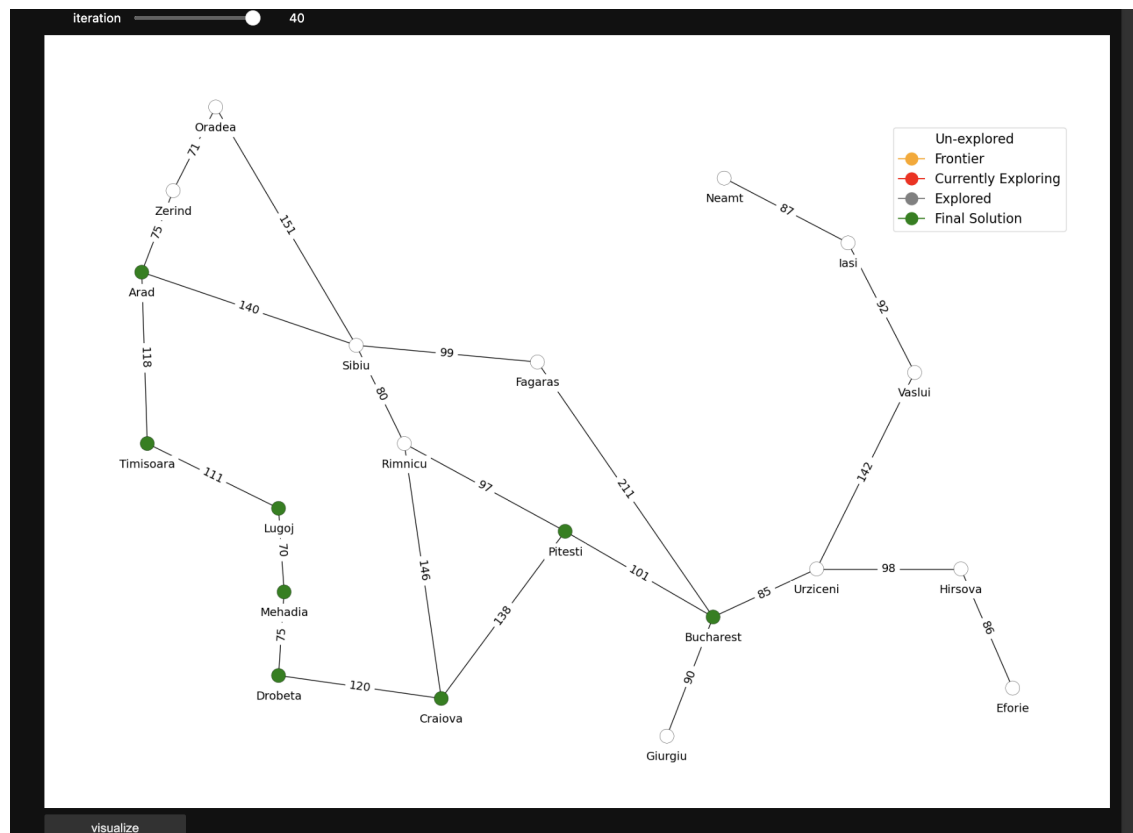
3.6 Analysis

- **Efficiency:** With 15 nodes expanded, BFS is quite efficient in exploring potential paths in this map scenario, particularly when the goal is not deep in the search tree. However, it may still explore many unnecessary nodes, especially if paths lead to cities with similar connections.

- **Optimality:** BFS guarantees the shortest path in terms of the number of edges, making it optimal for unweighted graphs. However, since it does not consider the varying travel costs, it may not yield the least-cost route among the nodes explored.
- **Complexity:** The time complexity of 4.1008×10^{-5} is relatively low compared to other algorithms. However, it has exponential space complexity, as it must keep track of all nodes at the current depth, which can be demanding in larger maps.

4 Depth-First Search (DFS)

4.1 Visualization



4.2 Path Found:

Arda $\rightarrow$ Timisoara $\rightarrow$ Lugoj $\rightarrow$ Mehadia $\rightarrow$ Drobeta $\rightarrow$ Craiova $\rightarrow$ Pitesti $\rightarrow$ Bucharest

Explanation: DFS dives deeper into a path before backtracking. It explores deeper nodes first, which may lead to a longer and costlier path. DFS does not guarantee finding the shortest or least-cost path. In this case, it follows a longer route across the western part of the map, resulting in a higher total travel cost compared to BFS.

4.3 The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:

```

1 import time
2
3 class Node:
4     def __init__(self, state, path_cost=0):
5         self.state = state
6         self.path_cost = path_cost # Initialize path cost
7
8     def expand(self, problem):
9         """Expand the node to generate children."""
10        children = []
11        for action, cost in problem.successors(self.state).items():
12            child_state = action
13            child_path_cost = self.path_cost + cost # Update the path cost for the child

```

```

14         children.append(Node(child_state, child_path_cost))
15     return children
16
17 class GraphProblem:
18     def __init__(self, initial, goal, graph):
19         self.initial = initial
20         self.goal = goal
21         self.graph = graph
22
23     def goal_test(self, state):
24         """Check if the current state is the goal state."""
25         return state == self.goal
26
27     def successors(self, state):
28         """Returns a dictionary of actions and costs to reach child states."""
29         return self.graph.get(state, {})
30
31 def depth_first_graph_search(problem):
32     """Search the deepest nodes in the search tree first."""
33
34     # Initialize metrics
35     iterations = 0
36     all_node_colors = []
37     node_colors = {k: 'white' for k in problem.graph.keys()}
38
39     # Start with the initial state
40     frontier = [Node(problem.initial)]
41
42     # Start timing the search
43     start_time = time.time()
44
45     while frontier:
46         # Popping the last node (depth-first)
47         node = frontier.pop()
48
49         # Modify the currently searching node to red
50         node_colors[node.state] = "red"
51         iterations += 1
52         all_node_colors.append(dict(node_colors))
53
54         # Check for goal state
55         if problem.goal_test(node.state):
56             node_colors[node.state] = "green" # Mark goal node as green
57             iterations += 1
58             all_node_colors.append(dict(node_colors))
59
60         # Calculate metrics
61         nodes_expanded = len(all_node_colors) # Number of nodes expanded
62         path_cost = node.path_cost # Path cost to the goal
63         time_elapsed = time.time() - start_time # Time taken for the search
64
65         return {
66             'nodes_expanded': nodes_expanded,
67             'path_cost': path_cost,
68             'time_complexity': time_elapsed,
69             'iterations': iterations,
70             'all_node_colors': all_node_colors,
71             'goal_node': node
72         }
73
74     # Expand the current node and add children to the frontier
75     frontier.extend(node.expand(problem))
76
77     # Modify the color of explored nodes to gray
78     node_colors[node.state] = "gray"
79     iterations += 1
80     all_node_colors.append(dict(node_colors))
81

```

```

82     # If no solution is found
83     return None
84
85 # Define the Romania problem
86 romania_map = {
87     'Arad': {'Sibiu': 140, 'Timisoara': 118},
88     'Sibiu': {'Arad': 140, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
89     'Timisoara': {'Arad': 118, 'Lugoj': 111},
90     'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
91     'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97},
92     'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101},
93     'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90},
94     'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
95     'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
96     'Drobeta': {'Mehadia': 75, 'Craiova': 120},
97     'Craiova': {'Drobeta': 120, 'Pitesti': 138},
98 }
99
100 # Instantiate the problem
101 romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)
102
103 # Run Depth First Search and get metrics
104 dfs_metrics = depth_first_graph_search(romania_problem)
105 if dfs_metrics:
106     print("Depth First Search Metrics:")
107     print("Nodes Expanded:", dfs_metrics['nodes_expanded'])
108     print("Path Cost:", dfs_metrics['path_cost'])
109     print("Time Complexity:", dfs_metrics['time_complexity'])
110 else:
111     print("No solution found.")

```

4.4 The Output:

```

# Depth First Search
node_colors = {k: 'white' for k in problem.graph.keys()}

# Start with the initial state
frontier = [Node(problem.initial)]

# Start timing the search
start_time = time.time()

while frontier:
    # Pop the last node (depth-first)
    node = frontier.pop()

    # Mark the currently searching node as red
    node_colors[node.state] = "red"
    iterations += 1
    all_node_colors.append(dict(node_colors))

    # Check for goal state
    if problem.goal_test(node.state):
        node_colors[node.state] = "green" # Mark goal node as green
        iterations += 1
        all_node_colors.append(dict(node_colors))

    # Calculate metrics
    nodes_expanded = len(all_node_colors) # Number of nodes expanded
    path_cost = node.path_cost # Path cost to the goal
    time_elapsed = time.time() - start_time # Time taken for the search

    return {
        'nodes_expanded': nodes_expanded,
        'path_cost': path_cost,
        'time_complexity': time_elapsed,
        'iterations': iterations,
        'all_node_colors': all_node_colors,
        'goal_node': node
    }

    # Expand the current node and add children to the frontier
    frontier.extend(node.expand(problem))

    # Mark the color of explored nodes to gray
    node_colors[node.state] = "gray"
    iterations += 1
    all_node_colors.append(dict(node_colors))

# If no solution is found
return None

# Define the Romania problem
romania_map = {
    'Arad': {'Sibiu': 140, 'Timisoara': 118},
    'Sibiu': {'Arad': 140, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Pitesti': 138},
}

# Instantiate the problem
romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)

# Run Depth First Search and get metrics
dfs_metrics = depth_first_graph_search(romania_problem)
if dfs_metrics:
    print("Depth First Search Metrics:")
    print("Nodes Expanded:", dfs_metrics['nodes_expanded'])
    print("Path Cost:", dfs_metrics['path_cost'])
    print("Time Complexity:", dfs_metrics['time_complexity'])
else:
    print("No solution found.")

Depth First Search Metrics:
Nodes Expanded: 16
Path Cost: 733
Time Complexity: 3.695487970674219e-05

```

4.5 Output Clarification

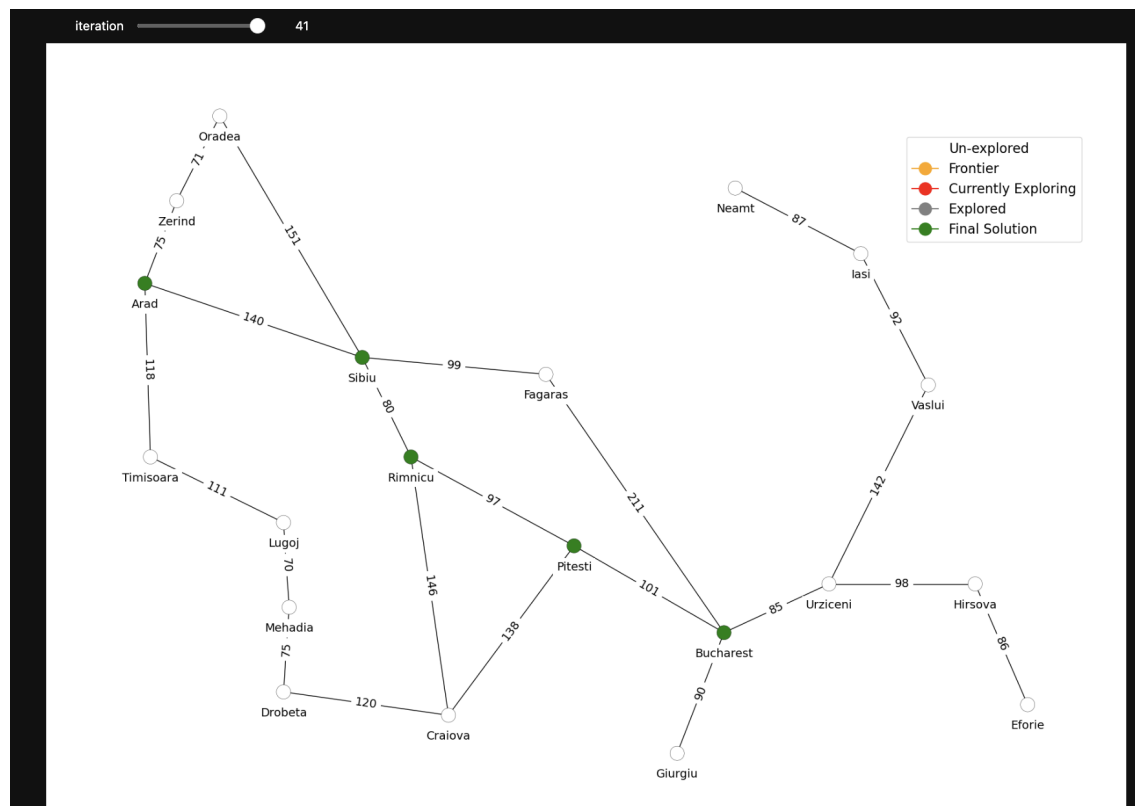
- Nodes Expanded: 16
- Total Cost: 733
- Time Complexity: $3.695487976074219e-05$

4.6 Analysis

- **Efficiency:** DFS expanded 16 nodes, slightly more than BFS, indicating a less efficient exploration process in this context. It often pursues deeper paths that may not lead to the goal efficiently, resulting in wasted explorations.
- **Optimality:** DFS is not optimal for finding the shortest or least-cost path. It may find a valid path, but the total cost can be high, as seen from its total cost of 733.
- **Complexity:** The time complexity of 3.6955×10^{-5} shows it can be quite quick in some scenarios, but this does not account for potentially infinite paths. DFS has lower memory requirements than BFS, but its tendency to follow deep paths can lead to inefficiencies in larger graphs.

5 Uniform Cost Search (UCS)

5.1 Visualization



5.2 Path Found:

Arad → Sibiu → Rimnicu → Pitesti → Bucharest

Explanation: UCS ensures the least-cost path by expanding the node with the lowest cumulative path cost. It explores more nodes than BFS and DFS to guarantee optimality in terms of cost. In this case, UCS finds a path that minimizes the total cost of travel, taking into account the varying distances between cities, resulting in a cost-effective route.

5.3 The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:

```
1 import time
2
3 def uniform_cost_search_graph(problem):
4     """Uniform Cost Search uses Best First Search algorithm with  $f(n) = g(n)$ ."""
5
6     # Start timing the search
7     start_time = time.time()
8
9     # Perform the search
10    iterations, all_node_colors, node = best_first_graph_search_for_vis(problem,
11                                                                           lambda node: node.path_cost)
12
13    # Calculate metrics
14    nodes_expanded = len(all_node_colors) # Number of nodes expanded
```

```

15 path_cost = node.path_cost if node else None # Path cost to the goal
16 time_elapsed = time.time() - start_time # Time taken for the search
17
18 return {
19     'nodes_expanded': nodes_expanded,
20     'path_cost': path_cost,
21     'time_complexity': time_elapsed
22 }
23
24 # Example usage
25 romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)
26
27 # Run Uniform Cost Search and get metrics
28 ucs_metrics = uniform_cost_search_graph(romania_problem)
29 print("Uniform Cost Search Metrics:", ucs_metrics)

```

Listing 2: Uniform Cost Search Algorithm with Metrics

5.4 The Output:

```

[161]: import time

def uniform_cost_search_graph(problem):
    """Uniform Cost Search uses Best First Search algorithm with f(n) = g(n)."""

    # Start timing the search
    start_time = time.time()

    # Perform the search
    iterations, all_node_colors, node = best_first_graph_search_for_vis(problem,
                                                                           lambda node: node.path_cost)

    # Calculate metrics
    nodes_expanded = len(all_node_colors) # Number of nodes expanded
    path_cost = node.path_cost if node else None # Path cost to the goal
    time_elapsed = time.time() - start_time # Time taken for the search

    return {
        'nodes_expanded': nodes_expanded,
        'path_cost': path_cost,
        'time_complexity': time_elapsed
    }

# Example usage
romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)

# Run Uniform Cost Search and get metrics
ucs_metrics = uniform_cost_search_graph(romania_problem)
print("Uniform Cost Search Metrics:", ucs_metrics)

Uniform Cost Search Metrics: {'nodes_expanded': 41, 'path_cost': 418, 'time_complexity': 0.0004551410675048828}

```

5.5 Output Clarification

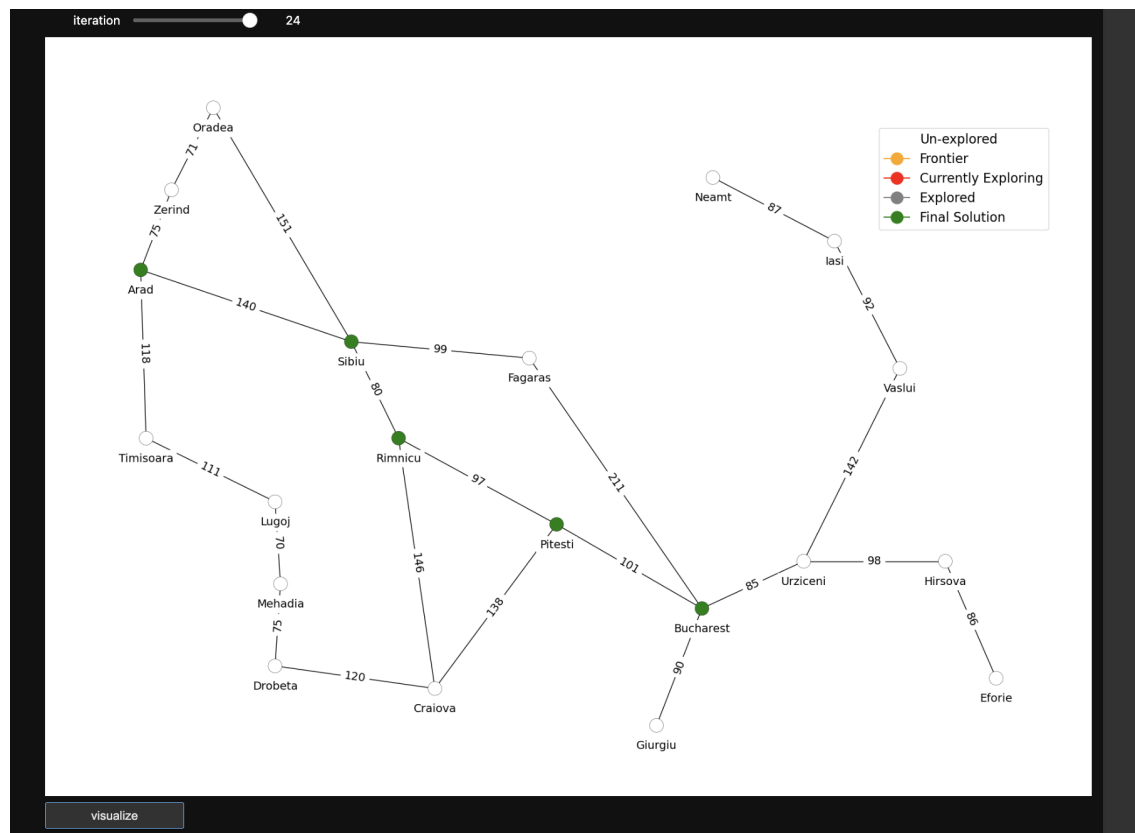
- Nodes Expanded: 41
- Total Cost: 418
- Time Complexity: 0.0004551410675048828

5.6 Analysis

- **Efficiency:** UCS expanded 41 nodes, significantly more than BFS and DFS, indicating that it explores many potential paths to ensure the least-cost solution. This could be seen as less efficient in terms of node expansion but is necessary to guarantee optimality.
- **Optimality:** With a total cost of 418, UCS is optimal, ensuring that the solution found is the least costly. It consistently prioritizes paths based on their cumulative cost, leading to optimal solutions.
- **Complexity:** The time complexity of 4.5514×10^{-4} reflects the additional time needed to evaluate the cost of various paths. UCS's memory consumption is high due to the need to maintain all expanded nodes, especially in dense networks with many connections.

6 A* Search

6.1 Visualization



6.2 Path Found

Arad → Sibiu → Rimnicu → Pitesti → Bucharest

Explanation: A* combines the advantages of UCS and heuristics, using both the actual path cost and an estimated cost to the goal (heuristic). This allows it to find the least-cost path while expanding fewer nodes than UCS. The path found by A* is identical to UCS in this case, as it also leads to the least-cost solution, but with potentially fewer nodes expanded.

6.3 The Code To Calculate Nodes Expanded, Total Cost, and Time Complexity:

```
1 import time
2
3 def astar_search_graph(problem, h=None):
4     h = memoize(h or problem.h, 'h')
5
6     # Start timing the search
7     start_time = time.time()
8
9     iterations, all_node_colors, node = best_first_graph_search_for_vis(problem,
10                                     lambda n: n.path_cost + h(n))
11
12     # Calculate metrics
```

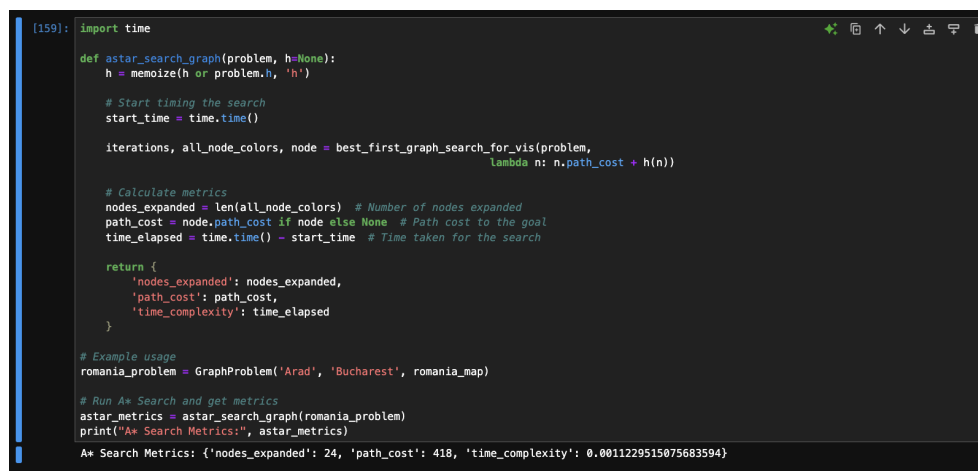
```

13 nodes_expanded = len(all_node_colors) # Number of nodes expanded
14 path_cost = node.path_cost if node else None # Path cost to the goal
15 time_elapsed = time.time() - start_time # Time taken for the search
16
17 return {
18     'nodes_expanded': nodes_expanded,
19     'path_cost': path_cost,
20     'time_complexity': time_elapsed
21 }
22
23 # Example usage
24 romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)
25
26 # Run A* Search and get metrics
27 astar_metrics = astar_search_graph(romania_problem)
28 print("A* Search Metrics:", astar_metrics)

```

Listing 3: A* Search Algorithm with Metrics

6.4 The Output:



```

[159]: import time

def astar_search_graph(problem, h=None):
    h = memoize(h or problem.h, 'h')

    # Start timing the search
    start_time = time.time()

    iterations, all_node_colors, node = best_first_graph_search_for_vis(problem,
                                                                           lambda n: n.path_cost + h(n))

    # Calculate metrics
    nodes_expanded = len(all_node_colors) # Number of nodes expanded
    path_cost = node.path_cost if node else None # Path cost to the goal
    time_elapsed = time.time() - start_time # Time taken for the search

    return {
        'nodes_expanded': nodes_expanded,
        'path_cost': path_cost,
        'time_complexity': time_elapsed
    }

# Example usage
romania_problem = GraphProblem('Arad', 'Bucharest', romania_map)

# Run A* Search and get metrics
astar_metrics = astar_search_graph(romania_problem)
print("A* Search Metrics:", astar_metrics)

A* Search Metrics: {'nodes_expanded': 24, 'path_cost': 418, 'time_complexity': 0.0011229515075683594}

```

6.5 Output Clarification

- Nodes Expanded: 24
- Total Cost: 418
- Time Complexity: 0.0011229515075683594

6.6 Analysis

- **Efficiency:** A* expanded 24 nodes, which is less than UCS but more than BFS and DFS. This indicates a good balance between exploration and cost evaluation. By utilizing heuristics, A* efficiently directs its search towards the goal, minimizing unnecessary expansions.
- **Optimality:** With a total cost of 418, A* is optimal, ensuring that it finds the least-cost path similar to UCS but typically with fewer node expansions due to heuristic guidance.
- **Complexity:** The time complexity of 1.1229×10^{-3} is higher than UCS but lower than many exhaustive searches. A* requires considerable memory as it maintains all paths being explored, but its heuristic-driven approach often leads to faster solutions.

7 Comparison Table

Algorithm	Nodes Expanded	Total Cost	Time Complexity
BFS	15	450	4.1008×10^{-5}
DFS	16	733	3.6955×10^{-5}
UCS	41	418	4.5514×10^{-4}
A*	24	418	1.1229×10^{-3}

Table 1: Comparison of Search Algorithms

8 Overall Comparison

In the context of the Romania Map Problem, BFS and DFS are quick but suboptimal, particularly in finding the least-cost path. UCS is optimal but less efficient in terms of node expansion due to the extensive exploration of paths. A* strikes a balance, providing optimal solutions while being more efficient in node expansion than UCS due to heuristic guidance. These analyses are critical in choosing the most suitable search algorithm for navigating the varied travel costs between Romanian cities.

9 Knowledge Acquired

In this assignment, I explored various search algorithms to tackle the Romania Map Problem, where the goal is to find the shortest path between cities while minimizing travel costs. Through this analysis, I gained a deeper understanding of four fundamental search algorithms: Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), and A* Search.

What I Learned:

- **Algorithm Characteristics:** Each algorithm has unique properties that make it suitable for different types of problems. For instance, BFS and DFS are more straightforward but may not yield optimal solutions, while UCS and A* provide optimal paths but with varying efficiency.
- **Complexity Analysis:** I learned how to evaluate the performance of these algorithms based on metrics such as nodes expanded, total cost, and time complexity.
- **Heuristic Utilization:** The importance of heuristics in improving the efficiency of search algorithms. A well-designed heuristic can significantly enhance the algorithm's performance in finding the shortest path.

Through this assignment, I have enhanced my problem-solving skills and deepened my understanding of search algorithms, equipping myself with valuable knowledge applicable to future projects in artificial intelligence and data structures.